

AWS PHP Application Hosting with ALB and Auto Scaling

1. Description

- 1.host a simple php web application in the aws instance
- 2.configure an Application load balacer for the instance containing web application.
- 3.create an auto scaling group with scaling policies for the application instance.
- 4.attach the load balancer with auto scaling group.
- 5.verify the working of scaling policies by increasing the load manually.

2. Environment and Resources Used

Resource	Configuration
AWS Region	Asia Pacific (Mumbai) – ap-south-1
Operating System	Red Hat Enterprise Linux 10.2
Instance Type	t3.micro
Launch Template	php-app-template
Application	PHP with Apache HTTP Server
Target Group	php-app-tg
Load Balancer	php-app-alb
Auto Scaling Group	php-app-asg
CPU Scaling Target	50%

Minimum / Desired / Maximum	1 / 1 / 2
-----------------------------	-----------

3. Creation of Security Groups

Two security groups were created to control network traffic between users, the load balancer, and the EC2 application instances.

3.1 ALB Security Group

The security group alb-sg was created for the Application Load Balancer.

Type	Port	Source
HTTP	80	Anywhere IPv4 (0.0.0.0/0)

The security group allowed users to access the application through the internet using HTTP.

3.2 EC2 Security Group

The security group webserver-sg was created for the EC2 application instances.

Type	Port	Source
HTTP	80	alb-sg
SSH	22	My IP

HTTP traffic was permitted only from the ALB security group, while SSH access was restricted to the user's public IP address.

4. Creation of the Launch Template

A Launch Template named `php-app-template` was created to define the configuration used to launch the application instances.

The template included the RHEL 10.2 AMI, `t3.micro` instance type, an existing key pair, `webserver-sg`, and a 10 GiB `gp3` root volume.

4.1 User Data Script

The following User Data script was added to the Launch Template to install Apache and PHP and create the application webpage.

```
#!/bin/bash
set -eux

# Install Apache, PHP, and curl
dnf install -y httpd php curl

# Enable and start Apache
systemctl enable --now httpd

# Get EC2 instance metadata using IMDSv2
TOKEN=$(curl -s -X PUT \
  "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

IP=$(curl -s \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  "http://169.254.169.254/latest/meta-data/local-ipv4")

HOSTNAME=$(curl -s \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  "http://169.254.169.254/latest/meta-data/hostname")

# Create the PHP webpage
cat > /var/www/html/index.php <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>AWS PHP Auto Scaling Lab</title>
</head>
```

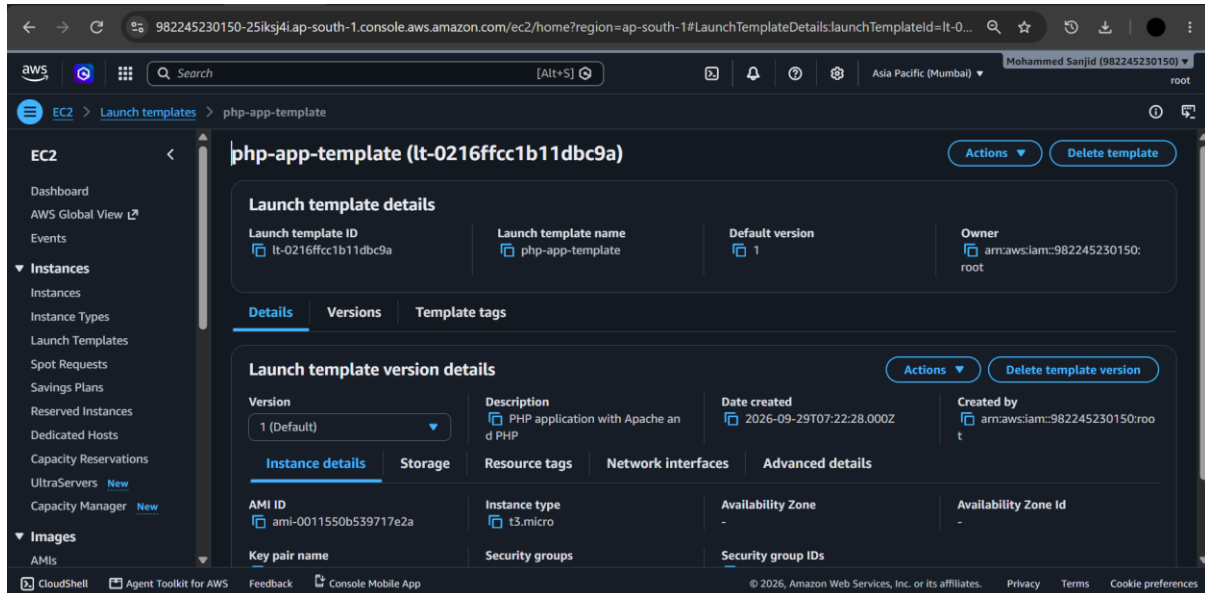
```
<body>
  <h1>Welcome to My AWS PHP Application</h1>
  <h2>EC2 Instance Details</h2>
  <p><strong>Private IP:</strong> $IP</p>
  <p><strong>Hostname:</strong> $HOSTNAME</p>
</body>
</html>
EOF
```

```
# Set file permissions
chown apache:apache /var/www/html/index.php
chmod 644 /var/www/html/index.php

# Restart Apache
systemctl restart httpd
```

The script automatically installed the required packages, enabled and started Apache, and created the PHP application page whenever an EC2 instance was launched using the template.

Screenshot 1: Launch Template Configuration



5. Creation of the Target Group

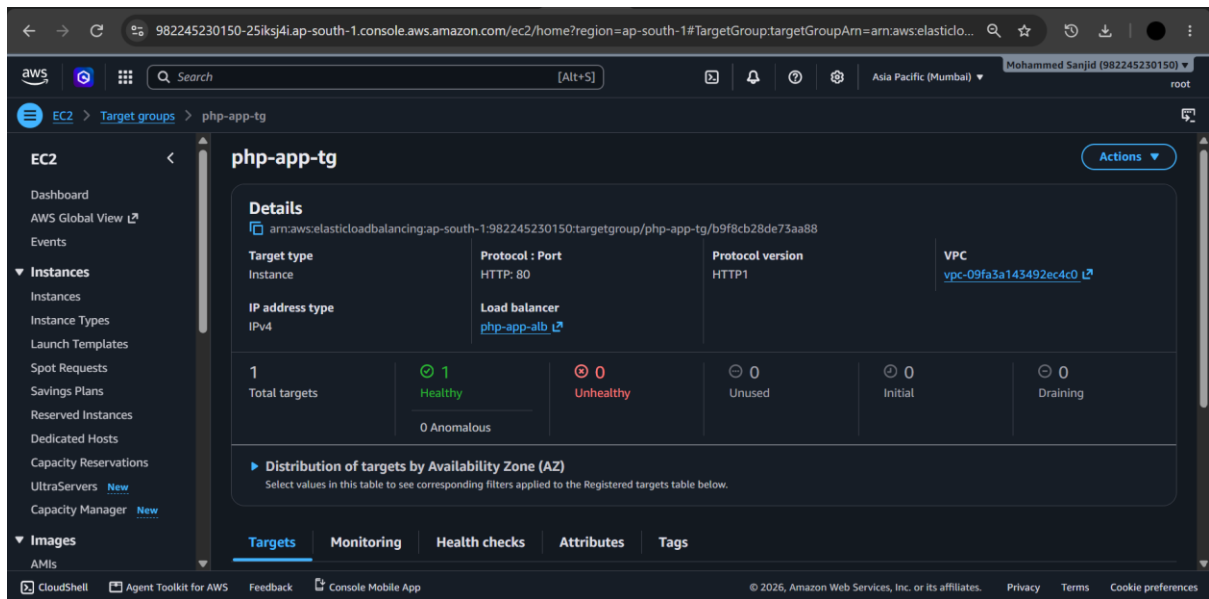
A target group named php-app-tg was created to register the EC2 instances and allow the ALB to forward incoming requests to them.

The target group was configured with the following settings:

Setting	Value
Target type	Instances
Protocol	HTTP
Port	80
Protocol version	HTTP1
Health check protocol	HTTP
Health check path	/
Success codes	200
Healthy threshold	2
Unhealthy threshold	2
Health check interval	30 seconds

The health check monitored whether the application instances were responding successfully. Instances passing the health checks were marked Healthy and became eligible to receive traffic.

Screenshot 2: Target Group and Healthy Target



6. Creation of the Application Load Balancer

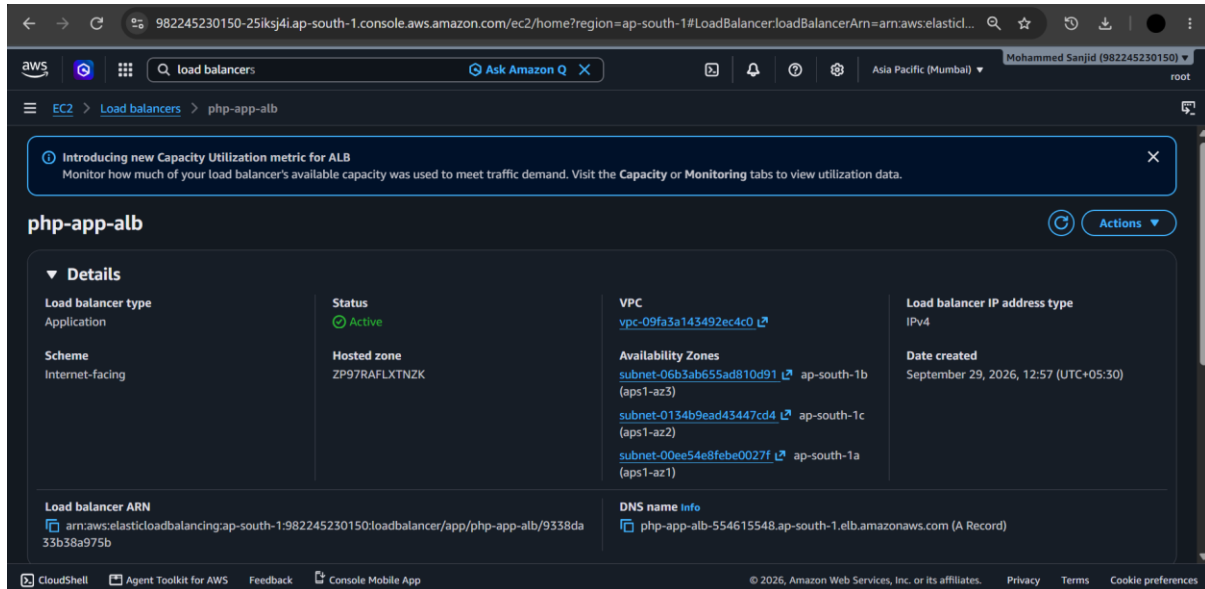
An internet-facing Application Load Balancer named `php-app-alb` was created to distribute incoming HTTP requests to the application instances.

The ALB was configured with the following settings:

Setting	Value
Scheme	Internet-facing
IP address type	IPv4
VPC	Default VPC
Availability Zones	Multiple Availability Zones
Security group	<code>alb-sg</code>
Listener	HTTP:80
Default action	Forward to <code>php-app-tg</code>

The ALB was configured to receive HTTP requests on port 80 and forward them to the registered instances in the target group.

Screenshot 3: Application Load Balancer Configuration



7. Creation of the Auto Scaling Group

An Auto Scaling Group named php-app-asg was created using php-app-template.

The ASG was configured to launch EC2 instances across the selected default subnets in multiple Availability Zones.

7.1 Auto Scaling Configuration

Setting	Value
Desired capacity	1
Minimum capacity	1
Maximum capacity	2
Health check type	EC2 and ELB

Health check grace period	300 seconds
---------------------------	-------------

The Auto Scaling Group was attached to the existing target group php-app-tg. This allowed newly launched instances to register automatically with the ALB.

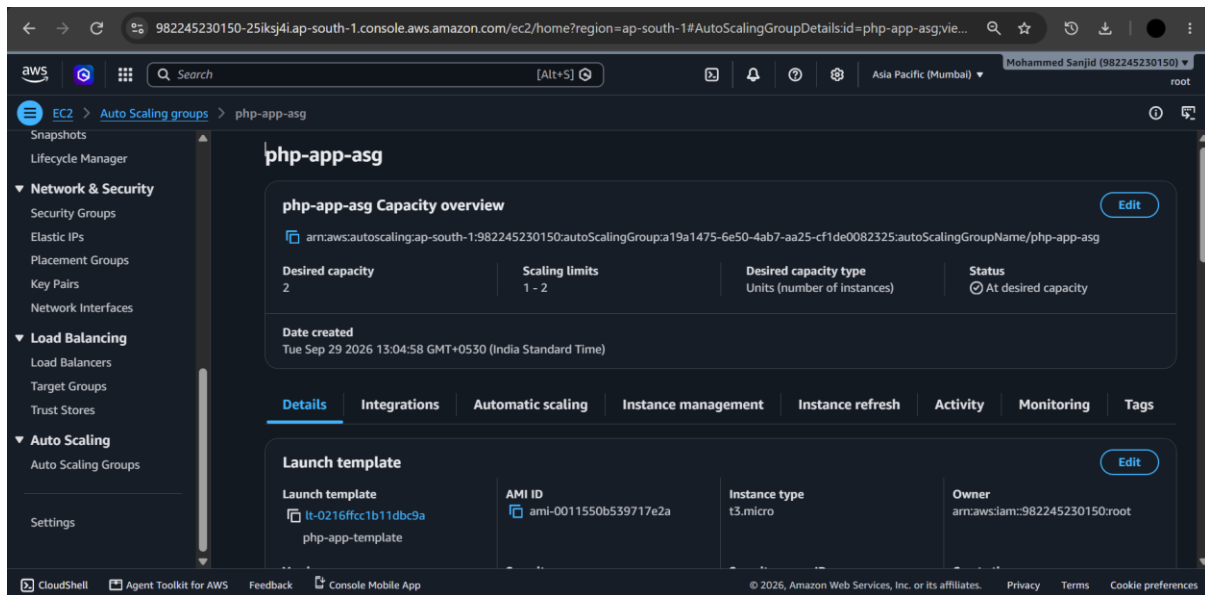
7.2 Scaling Policy

A target tracking scaling policy was configured to maintain average CPU utilization at 50%.

Setting	Value
Policy type	Target tracking
Metric	Average CPU utilization
Target value	50%
Instance warmup	300 seconds
Scale-in	Enabled

The scaling policy allowed the ASG to increase or decrease the number of EC2 instances according to CPU utilization, within the configured minimum and maximum capacity.

Screenshot 4: Auto Scaling Group Configuration and Scaling Policy



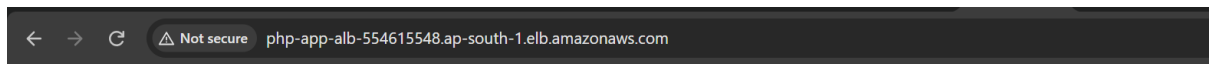
8. Verification of the PHP Application Through the ALB

After creating the ALB and ASG, the ALB DNS name was opened in a web browser.

The PHP application webpage loaded successfully and displayed the welcome message and EC2 instance details, including the private IP address and hostname.

The webpage was also refreshed multiple times, and the displayed instance IP address changed. This demonstrated that requests were being served by different EC2 instances through the load balancer.

Screenshot 5: PHP Application Accessible Through the ALB

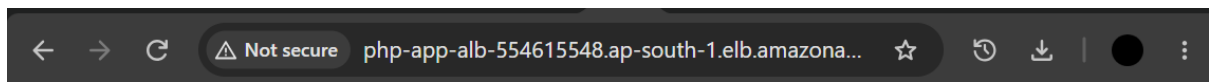


Welcome to My AWS PHP Application

EC2 Instance Details

Private IP: 172.31.11.207

Hostname: ip-172-31-11-207.ap-south-1.compute.internal



Welcome to My AWS PHP Application

EC2 Instance Details

Private IP: 172.31.21.75

Hostname: ip-172-31-21-75.ap-south-1.compute.internal

9. Manual CPU Load and Auto Scaling Test

The Auto Scaling functionality was verified by manually increasing CPU utilization on the EC2 instance.

9.1 Installation and Verification of stress-ng

The stress-ng utility was downloaded and installed on the RHEL instance. Its installed version was verified using:

```
stress-ng --version
```

The command returned version 0.19.03.

9.2 Generating CPU Load

The following command was executed through SSH to generate CPU load:

```
sudo stress-ng --cpu 2 --timeout 600s
```

The command started two CPU stress workers for 600 seconds.

During the test, the top command was used to monitor CPU utilization. CPU usage reached approximately 100%.

Screenshot 6: CPU Utilization at 100% During the Stress Test

```
[ec2-user@ip-172-31-21-75 ~]$ top
top - 08:35:06 up 59 min,  3 users,  load average: 2.00, 1.69, 0.95
Tasks: 135 total,  3 running, 132 sleeping,  0 stopped,  0 zombie
%Cpu(s): 99.5 us,  0.2 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.3 hi,  0.0 si,  0.0 st
MiB Mem :  906.8 total,  183.8 free,  434.3 used,  438.2 buff/cache
MiB Swap:   0.0 total,   0.0 free,   0.0 used.  472.5 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2376	root	20	0	70016	8576	4240	R	99.3	0.9	6:13.51	stress-ng-cpu
2377	root	20	0	70016	8576	4240	R	99.3	0.9	6:13.24	stress-ng-cpu
36	root	20	0	0	0	0	I	0.3	0.0	0:00.06	kworker/u8:2-events_unbound
749	chronyd	20	0	10600	4984	4364	S	0.3	0.5	0:00.09	chronyd
1	root	20	0	44420	36240	10460	S	0.0	3.9	0:03.08	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0.0	0.0	0:00.75	kworker/u8:1-flush-259:0
15	root	20	0	0	0	0	S	0.0	0.0	0:00.01	ksoftirqd/0
16	root	20	0	0	0	0	I	0.0	0.0	0:00.08	rcu_preempt
17	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_work+
18	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_gp_kthread_worker
19	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/0

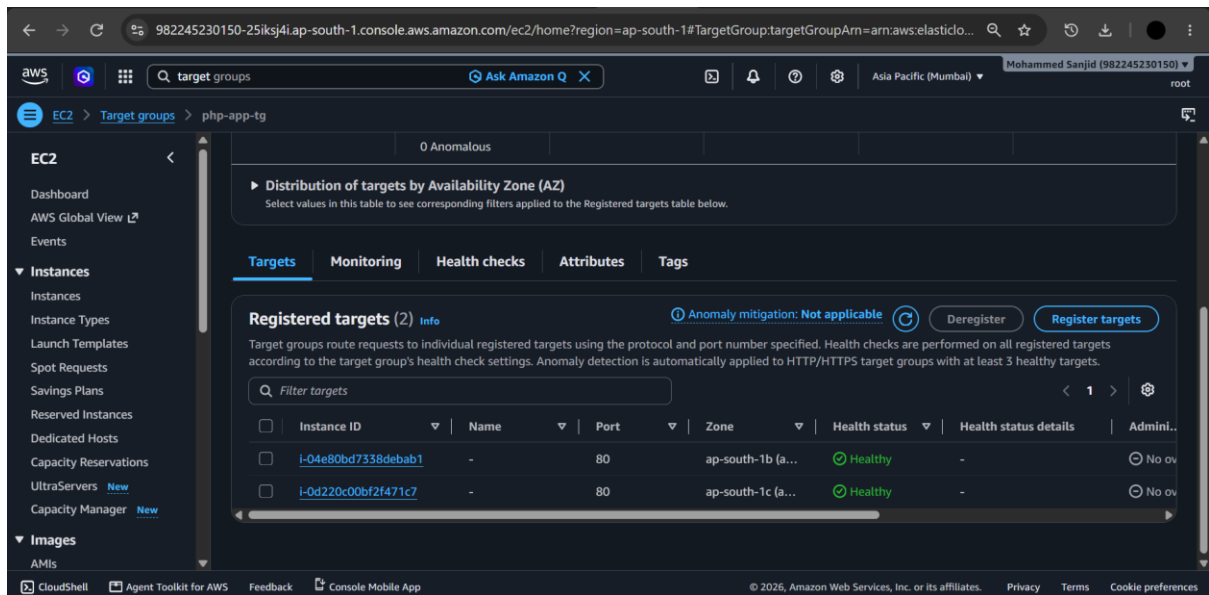
9.3 Verification of Scale-Out

While the CPU load was being generated, the target group was checked in the AWS Console.

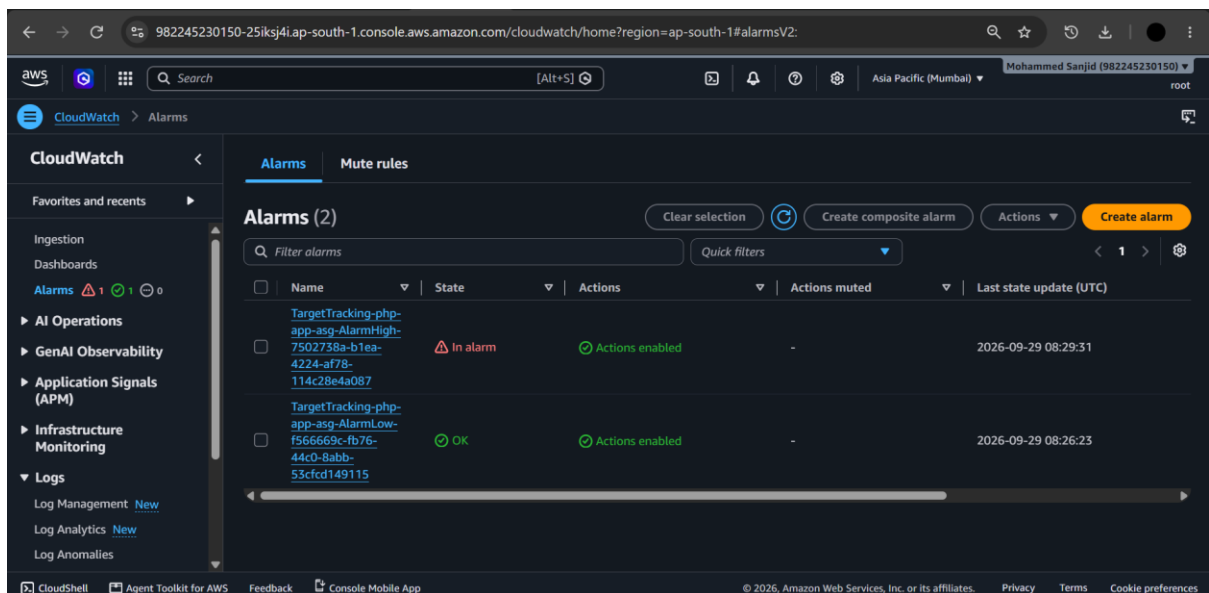
Two instances appeared in the target group during the scaling test. The second instance was launched by the Auto Scaling Group in response to the increased CPU load.

The CloudWatch alarm also displayed a warning state during the high CPU utilization period.

Screenshot 7: Two Instances Registered During Scale-Out



Screenshot 8: CloudWatch Alarm in Warning State



10. Verification of Scale-In and Recovery

After observing the high CPU utilization and the additional instance, the stress test was stopped.

CPU utilization was checked again using:

top

After terminating stress-ng, CPU utilization dropped to less than 1%.

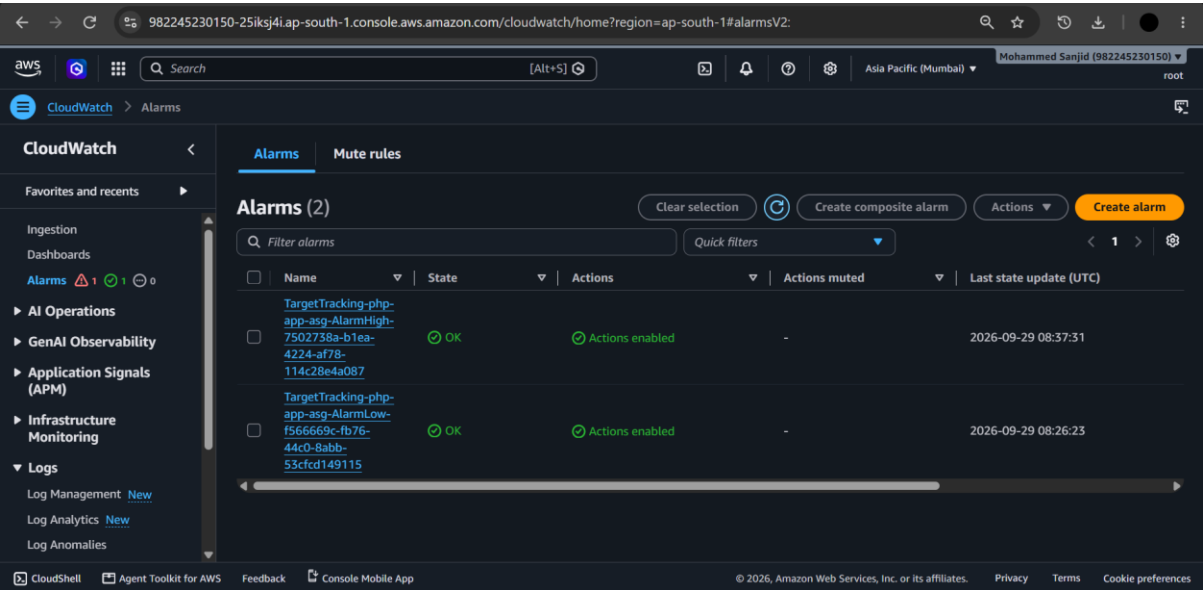
The CloudWatch alarm subsequently changed from the warning state to the OK state, indicating that the monitored CPU metric had returned to the alarm's normal range.

Screenshot 9: CPU Utilization Below 1% After Stopping the Stress Test

```
[ec2-user@ip-172-31-21-75 ~]$ top
top - 08:48:24 up 1:13, 3 users, load average: 0.00, 0.12, 0.41
Tasks: 129 total, 1 running, 128 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.2 us, 0.0 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 906.8 total, 215.6 free, 402.3 used, 402.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 504.5 avail Mem

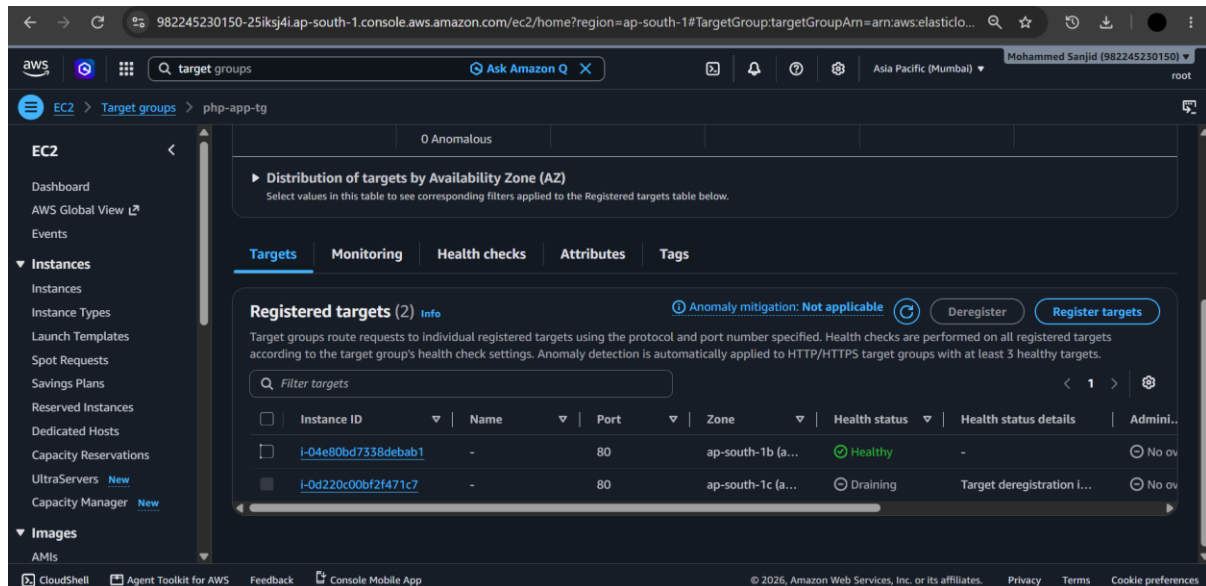
  PID USER      PR  NI    VIRT    RES    SHR  S  %CPU  %MEM    TIME+  COMMAND
 2498 ec2-user  20   0   7808    5532   3296  R   0.3   0.6   0:00.76 top
    1 root      20   0   44420   36240  10460  S   0.0   3.9   0:03.19 systemd
    2 root      20   0         0         0         0  S   0.0   0.0   0:00.00 kthreadd
    3 root      20   0         0         0         0  S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
    7 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
    8 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-netns
   10 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
   13 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
   15 root      20   0         0         0         0  S   0.0   0.0   0:00.01 ksoftirqd/0
   16 root      20   0         0         0         0  I   0.0   0.0   0:00.09 rcu_preempt
   17 root      20   0         0         0         0  S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
   18 root      20   0         0         0         0  S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
   19 root      rt   0         0         0         0  S   0.0   0.0   0:00.00 migration/0
   20 root     -51   0         0         0         0  S   0.0   0.0   0:00.00 idle_inject/0
   21 root      20   0         0         0         0  S   0.0   0.0   0:00.00 cpuhp/0
   22 root      20   0         0         0         0  S   0.0   0.0   0:00.00 cpuhp/1
   23 root     -51   0         0         0         0  S   0.0   0.0   0:00.00 idle_inject/1
   24 root      rt   0         0         0         0  S   0.0   0.0   0:00.04 migration/1
   25 root      20   0         0         0         0  S   0.0   0.0   0:00.02 ksoftirqd/1
   27 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
   29 root      20   0         0         0         0  S   0.0   0.0   0:00.00 kdevtmpfs
   30 root      0 -20         0         0         0  I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
   31 root      20   0         0         0         0  I   0.0   0.0   0:00.00 rcu_tasks_kthread
   32 root      20   0         0         0         0  I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   33 root      20   0         0         0         0  I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
   34 root      20   0         0         0         0  S   0.0   0.0   0:00.00 kauditd
   35 root      20   0         0         0         0  S   0.0   0.0   0:00.00 khungtaskd
   36 root      20   0         0         0         0  I   0.0   0.0   0:00.08 kworker/u8:2-events_unbound
   37 root      20   0         0         0         0  S   0.0   0.0   0:00.00 oom_reaper
```

Screenshot 10: CloudWatch Alarm Returned to OK



The Auto Scaling Group was then monitored while it returned toward its configured desired capacity of one instance. The additional instance was observed entering termination.

Screenshot 11: Auto Scaling Activity and Scale-In



11. Conclusion

The PHP web application was successfully hosted on an AWS EC2 instance using Apache and PHP. An Application Load Balancer was configured to distribute incoming HTTP requests to the registered application instances, and the Auto Scaling Group was attached to the target group with a target tracking CPU scaling policy.

The manual CPU load test using `stress -ng` demonstrated increased CPU utilization, the appearance of an additional instance during scale-out, and the recovery of CPU utilization after the stress test was stopped. CloudWatch alarm states were also observed changing from warning to OK.

The assignment demonstrated the practical implementation of PHP application hosting, load balancing, health checks, automatic scaling, and CPU monitoring using AWS services.